

Execution-Aware A* Search for Cross-Exchange Stablecoin Arbitrage

Kevin Litvin

Hang Ma

Simon Fraser University

(emails omitted for review)

Abstract

Cross-exchange cryptocurrency arbitrage enables low-risk profit from price discrepancies across exchanges, yet existing approaches predominantly employ negative cycle detection algorithms (e.g., Bellman–Ford) that target opportunity identification rather than execution feasibility. We introduce an execution-aware pathfinding framework for stablecoin cross-exchange arbitrage using A* search with domain-specific guidance heuristics. The problem is modelled as a weighted directed graph where nodes represent (exchange, stablecoin) pairs across 12 major centralized exchanges (41 nodes, 864 edges) and edges encode trades and transfers with real-world execution costs—trading fees, order-book slippage, withdrawal fees, gas costs, transfer delays, and exchange reliability risks. Stablecoins, assets pegged to fiat currencies such as the US dollar, provide the trading medium due to their low volatility and high cross-venue liquidity. We propose three domain-specific guidance heuristics— h_1 (liquidity), h_2 (slippage), h_4 (chain congestion and exchange risk via Weighted A*)—and a multi-start search-control strategy that diversifies exploration across starting exchanges. Evaluation over 7,200 search instances in an 8-hour overnight campaign, graph scaling from 4–12 exchanges, sensitivity sweeps across four parameter dimensions, and cached-graph benchmarks from 30 start nodes reveals differentiated heuristic roles: h_2 reduces node expansions by 29% relative to Dijkstra while matching profit, and h_4 provides risk-aware guidance at Dijkstra-class latency (P99 = 33 ms) with zero API overhead. The execution-aware graph formulation itself is the primary contribution: Dijkstra alone achieves 100% success across overnight runs, demonstrating that embedding real-world frictions into edge weights makes any shortest-path algorithm a viable arbitrage planner.

1. Introduction

1.1. Research Questions and Problem Statement

Cross-exchange stablecoin arbitrage exploits price differences for the same asset across centralized exchanges. While the idea appears straightforward—buy on the cheaper exchange and sell on the more expensive one—practical execution is significantly more complex. Profitable opportunities depend not only on visible price spreads, but also on fees, slippage, transfer delays, liquidity constraints, exchange reliability, and the short time windows in which arbitrage remains viable. Manually identifying profitable multi-exchange trading cycles is slow, error-prone, and not scalable.

This project formulates cross-exchange stablecoin arbitrage as a graph-planning problem and develops an automated route planner that computes complete, executable trade-and-transfer cycles rather than simple pairwise price gaps. By incorporating trading fees, withdrawal costs, transfer latencies, liquidity depth, and live market data via CCXT, the system generates execution-ready arbitrage paths. Unlike prior tools such as Crypto Advisor (Oantă & Coroiu, 2023), which detect price discrepancies without modeling full execution cycles or risk, our approach integrates real-world constraints and applies A* search with domain-specific heuristics.

Our methodology is structured around several research questions. First, do certain heuristics consistently outperform others on specific classes of trading instances? We examine whether h_4 (chain congestion + exchange reliability) dominates due to richer risk modeling, whether h_1 (liquidity) performs best in high-volume markets but struggles in sparse ones, whether h_2 (slippage) excels when order-book structure is the primary bottleneck, and whether h_3 (parallel multi-start) can discover profitable opportunities missed by single-start A*.

Second, how does each heuristic influence A*'s node-expansion behavior? Because heuristics determine search order, understanding expansion bias is critical. For example, h_1 prioritizes deep markets, potentially skipping thin but profitable routes. h_2 penalizes shallow order books and

PLEASE get rid of the em-dash :-)

ok ✓

Gift-core

#I have a question: why did you rearmy the heuristics?

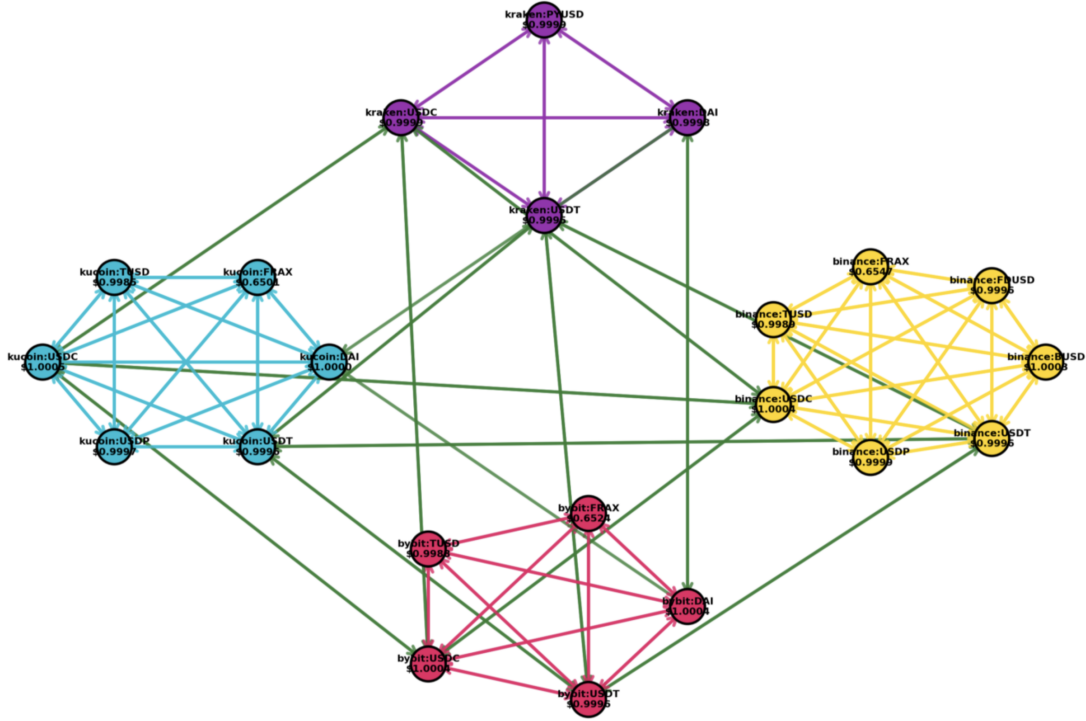


Figure 1. Overview of the cross-exchange arbitrage graph used in our experiments. Nodes represent (exchange, asset) pairs. Intra-exchange edges denote trading pairs, while inter-exchange edges represent asset transfers.

favors conservative execution paths. h_4 deprioritizes fast-but-risky blockchains or unreliable exchanges, trading theoretical profit for robustness. In contrast, h_3 diversifies exploration by expanding multiple regions in parallel.

Third, how does algorithmic performance change as structural difficulty increases? Hard instances arise from liquidity sparsity, limited chain overlap, high withdrawal fees (e.g., Ethereum), short arbitrage windows, and fragmented exchange listings. These constraints produce complex search landscapes analogous to bottleneck-heavy planning problems.

Finally, when heuristics return suboptimal routes, how close are they to the best-known solution? We compare achieved profit against the maximum observed across all heuristics, assess whether certain heuristics consistently produce near-optimal results, evaluate whether Weighted A*'s safety bias reduces profitability, and determine whether parallel multi-start mitigates weaknesses of single-heuristic searches. We also identify failure patterns where certain heuristics degrade under increased order size or path depth.

1.2. Why Cross-Exchange Stablecoin Arbitrage Matters

Stablecoins serve as the primary liquidity bridges in cryptocurrency markets, connecting exchanges, blockchains,

and trading pairs. Because centralized exchanges operate independently, price discrepancies for the same USD-pegged asset frequently arise due to fragmented liquidity, withdrawal delays, blockchain congestion, regional demand asymmetries, and operational frictions. These deviations are typically small in magnitude but persistent in structure, forming a dense and highly connected cross-exchange network.

Unlike volatile cryptocurrencies, stablecoins operate within a bounded price band anchored to fiat currency. This bounded deviation property reframes arbitrage from speculative price prediction into a constrained execution-optimization problem. Profitability is determined not by directional market movement, but by the ability to coordinate trades and transfers while minimizing fees, slippage, latency, and risk. This creates a uniquely structured planning domain in which path-dependent execution costs dominate expected return.

During periods of macroeconomic instability or crypto-market stress, stablecoin flows intensify as capital migrates toward dollar-pegged assets. These periods amplify temporary cross-exchange imbalances, increasing both spread frequency and execution complexity. As a result, the structured stablecoin arbitrage graph becomes particularly valuable during economic volatility, where efficient routing of dollar-denominated liquidity provides both prof-

you are with this paragraph block, i & contextualization

last sentence might be a tiny bit (redundant), correct?

itability opportunities and market-stabilizing effects. The dataset therefore captures economically meaningful liquidity movements rather than transient speculative signals.

1.3. Key Challenges (Liquidity, Slippage, Latency, Reliability)

Cross-exchange arbitrage is constrained by several real-world frictions that significantly complicate route planning.

Liquidity. Order-book depth determines how much capital can be deployed without moving the market. Thin markets may display attractive spreads but cannot support meaningful trade volume without eroding profit.

Slippage. Execution prices depend on order-book structure. Large orders incur nonlinear price impact, meaning theoretical profit often differs from realized profit.

Latency. Cross-exchange transfers require blockchain confirmations, and arbitrage windows may close before assets arrive. Chain congestion and confirmation delays introduce time-dependent risk.

Reliability. Exchanges may experience withdrawal suspensions, API instability, or execution failures. Some blockchains offer faster transfers but higher operational risk.

These constraints interact in complex ways. For example, faster chains may reduce latency but carry higher reliability risk, while deep liquidity markets may impose higher trading fees. Effective arbitrage planning therefore requires balancing profit maximization with robustness under uncertainty. Our heuristic-guided A* framework explicitly models these trade-offs to prioritize routes that are not only theoretically profitable but realistically executable.

2. Related Work

Cryptocurrency arbitrage has been studied from multiple perspectives, including graph-theoretic opportunity detection, system-level arbitrage platforms, and execution strategy optimization. Most prior work focuses primarily on identifying arbitrage opportunities, while comparatively less attention has been devoted to modeling execution feasibility under real-world constraints. This section situates our approach within these research streams.

2.1. Graph-Theoretic Arbitrage Detection (Negative Cycles, Bellman-Ford)

The dominant formulation of arbitrage detection models financial markets as directed weighted graphs. Nodes represent currencies or tokens, and exchange rates are encoded as edges. Applying a logarithmic transformation,

$$w(e) = -\log(r(e)),$$

converts multiplicative exchange rates into additive costs, allowing arbitrage opportunities to be identified as negative-

weight cycles. This transformation enables the use of classical shortest-path algorithms such as Bellman-Ford and its variants.

Oantă and Coroiu (2023) adopt this paradigm for cross-exchange cryptocurrency markets in their system *Crypto Advisor*, which aggregates real-time exchange data and detects arbitrage cycles using graph-based methods. The primary contribution is infrastructural, providing a scalable web-based platform for reporting opportunities. However, the detection framework assumes idealized execution conditions, including sufficient liquidity, static prices during execution, and negligible operational risk.

Similar graph-based formulations appear in decentralized finance (DeFi) arbitrage systems, improved negative-cycle detection algorithms, and real-time high-frequency frameworks. Across these works, the core objective is opportunity existence detection rather than execution-aware feasibility analysis.

2.2. Pathfinding Approaches in Financial Routing

Beyond negative-cycle detection, shortest-path formulations have been proposed to compute cost-minimizing trading routes. Algorithms such as Dijkstra's method and its weighted variants provide efficient routing in additive cost graphs. However, these approaches typically assume static and deterministic edge weights.

In cross-exchange contexts, routing decisions involve heterogeneous constraints, including withdrawal fees, transfer costs, and asset availability across exchanges. Traditional shortest-path algorithms can model additive costs but do not incorporate domain-specific risk factors or execution uncertainty during search. As a result, routing decisions often remain detached from real-world feasibility.

2.3. Heuristic Search in Constrained Optimization

Heuristic search methods such as A* have been widely studied in artificial intelligence for solving constrained pathfinding and planning problems. By incorporating domain-specific heuristics, A* can guide search toward promising regions of the state space while preserving optimality under admissible conditions.

Although heuristic search has been extensively applied in robotics, logistics, and planning domains, its application to cross-exchange cryptocurrency arbitrage remains limited. Prior financial graph formulations focus on detecting negative cycles or computing shortest paths without embedding execution risk directly into heuristic guidance. This creates an opportunity to apply AI planning techniques to arbitrage execution planning under real-world constraints.

source?

kill

also need source here, literally any example paper should be ok

kill

keep

2.4. Arbitrage Under Latency, Fees, and Market Frictions

Execution feasibility has been studied more extensively in single-exchange optimal execution literature, often using reinforcement learning or stochastic control methods (e.g., Kearns & Nevmyvaka, 2006). These approaches optimize trade scheduling within a single market, accounting for price impact and liquidity dynamics.

However, cross-exchange arbitrage introduces additional constraints, including blockchain transfer latency, withdrawal fees, liquidity fragmentation, and exchange reliability. Stablecoin markets, in particular, exhibit narrow spreads and frequent micro-inefficiencies, making them highly sensitive to execution frictions. Existing detection-oriented methods do not explicitly integrate these constraints into the search process, increasing the risk of theoretical but non-executable opportunities.

2.5. Comparison to System-Oriented Arbitrage Platforms

Several systems-oriented contributions focus on scalable architectures for aggregating exchange data, computing spreads, and streaming arbitrage signals through APIs or dashboards. These works emphasize infrastructure, latency reduction, and data engineering robustness.

While valuable from a systems perspective, such platforms typically treat arbitrage computation as a spread comparison or post-hoc cycle detection task. Execution constraints—including liquidity depth, slippage, transfer delays, and operational risk—are evaluated after opportunity detection rather than embedded within the search itself.

2.6. Risk-Aware A* and Stochastic Pathfinding

A growing body of work extends classical A* to settings where edge costs are uncertain or drawn from probability distributions. Nikolova and Karger (2008) study risk-averse pathfinding under stochastic edge weights using first-order stochastic dominance (FSD) as a preference criterion, demonstrating that risk-averse shortest paths can differ substantially from expected-cost shortest paths. More recently, Lim et al. (2022) propose Conditional Value-at-Risk (CVaR) pathfinding, where the objective is to minimize the worst-case tail cost of a path, relevant to scenarios where downside risk is more important than average performance.

Our heuristic h_4 (chain congestion and exchange reliability) implicitly addresses a similar concern: by penalizing paths through unreliable exchanges or congested chains, we steer the search toward routes with lower variance in execution outcomes. Future work could formalize this connection by adopting CVaR or FSD criteria directly in the edge-cost model.

2.7. Cross-Chain Arbitrage and Bridging Empirics

Empirical studies of cross-chain arbitrage execution have documented the importance of bridging latency and inventory management. Obadia et al. (2021) analyze flashbots and maximal extractable value (MEV), showing that on-chain arbitrage profits are highly sensitive to transaction ordering and confirmation latency. In the cross-exchange (CEX-to-CEX) setting, withdrawal processing times and blockchain finality delays introduce additional uncertainty that is absent in single-chain MEV extraction.

Our transfer-time model and chain reliability heuristic (h_4) draw on these insights: we incorporate blockchain finality times (ranging from ~ 1 s for Solana to ~ 600 s for Ethereum L1) and exchange processing overhead into the search, ensuring that discovered paths are time-feasible under realistic settlement conditions.

2.8. Liquidity and Slippage Models in CEX and AMM Markets

The relationship between order size and execution cost has been extensively modeled in both centralized exchange (CEX) and automated market maker (AMM) settings. In AMMs, slippage follows a deterministic curve defined by the bonding function (e.g., the constant-product formula $x \cdot y = k$), yielding quadratic cost in order size for small trades. In CEX order books, slippage depends on the depth profile of the limit order book and can exhibit discontinuities at price levels with thin liquidity.

Our heuristic h_2 uses a simplified model: we walk the live order book to compute VWAP for a given order size and convert the price impact to basis points. This approach captures the essential structure of order-book slippage without requiring parametric model fitting. We acknowledge that more sophisticated piecewise or quadratic slippage models (as used in optimal execution literature) could improve accuracy for very large orders, and note this as a direction for future work.

2.9. Positioning of Our Contribution

In contrast to prior work, our approach integrates feasibility modeling directly into the search process. Instead of first detecting theoretical arbitrage and then evaluating execution risk, we formulate arbitrage planning as a constraint-aware search problem and apply A* with domain-specific guidance heuristics. This approach bridges opportunity detection and executable route construction, particularly suited to the low-margin, friction-sensitive context of stablecoin markets.

overall, spending 1.5 pgs on related work is excessive! i think keep only 2.1, 2.4, 2.6, 2.7 as is, and consolidate 2.2, 2.3, 2.8 into a "research gaps" or whatever name is appropriate section before 2.9
and 2.5

3. Problem Formulation

3.1. Graph Representation

We formulate the stablecoin cross-exchange arbitrage problem as a directed weighted graph $G = (V, E)$. Each node $v \in V$ represents a unique stablecoin–exchange pair, written as $v = (\text{exchange}, \text{coin})$. Each directed edge $e \in E$ corresponds either to a trade executed on an exchange or a transfer of funds between exchanges.

Every edge has an associated effective exchange rate that incorporates all relevant costs—including trading fees, withdrawal fees, spreads, and slippage. We assign each edge a weight using the negative logarithm of this effective rate:

$$w(e) = -\log(\text{effective_rate}(e)).$$

This transformation converts multiplicative returns into additive costs, enabling the use of shortest-path algorithms to evaluate profitability.

3.2. Path Profitability

Given a path $P = (v_0, v_1, \dots, v_k)$ with initial capital C_0 , the final capital after executing all trades and transfers along the path is

$$C_k = C_0 \cdot \prod_{e \in P} r(e)$$

where $r(e)$ denotes the effective rate on edge e . A path is profitable when the net gain exceeds a user-specified minimum profit threshold:

$$C_k - C_0 \geq \text{min_profit}.$$

Expressing this condition in additive log space gives an equivalent constraint:

$$\sum_{e \in P} -\log(r(e)) \leq -\log\left(1 + \frac{\text{min_profit}}{C_0}\right).$$

Thus, the arbitrage search problem becomes the task of identifying a path that begins at a given start node v_0 and ends at some end node v_k , and yields a multiplicative return greater than one. More formally, we seek:

$$\text{Find } P : v_0 \rightarrow v_k \text{ s.t. } \prod_{e \in P} r(e) > 1$$

This constitutes a minimum-cost path search under realistic execution constraints.

Goal Condition. We define the *goal set* $\mathcal{G}$ as the set of all nodes reachable from the start node v_0 at which the accumulated capital exceeds the initial capital:

$$\mathcal{G} = \{v_k \in V \mid C_k = C_0 \cdot \prod_{e \in P} r(e) > C_0\}.$$

Critically, we do *not* require $v_k = v_0$. This is an **open-path** formulation: the trader may end on any exchange–coin pair where the marked-to-market USD value of the position exceeds the initial capital.

This design is economically justified because all assets in our graph are stablecoins pegged to USD. The terminal position $v_k = (\text{exchange}_k, \text{coin}_k)$ holds a stablecoin whose value is within a narrow band of \$1.00 by construction (we enforce a price tolerance of $\pm 2\%$ when building the graph). Therefore, “profit” at any terminal node is realized value: the trader holds C_k units of a USD-pegged asset, and $C_k - C_0$ is the net gain. No additional conversion step is needed to “close” the position into USD.

Standard arbitrage literature typically requires closed cycles ($v_k = v_0$), but this restriction is unnecessarily limiting in the stablecoin domain. A profitable open path—for example, starting with USDT on Binance and ending with USDC on Kraken—yields the same economic outcome as a closed cycle, because both endpoints hold fungible USD-denominated value. Our formulation therefore maximizes the opportunity set without sacrificing profit interpretation.

3.3. A* Search Formulation

To solve the arbitrage problem efficiently, we apply the A* search algorithm. For each node n , we maintain a cost-to-reach value $g(n)$, computed as the sum of the additive edge costs encountered so far:

$$g(n) = \sum_{e \in \text{path}} -\log(r(e)).$$

We define a heuristic function $h(n)$ that estimates the remaining cost or execution risk required to complete a profitable cycle. The A* evaluation function is given by

$$f(n) = g(n) + h(n),$$

where $g(n)$ denotes the accumulated path cost in log space and $h(n)$ estimates the remaining cost to complete a feasible arbitrage path. The algorithm terminates when a path yields final capital exceeding the initial capital, that is, when $C_n > C_0$. This formulation allows A* to balance realized execution costs with heuristic estimates of remaining feasibility and risk.

Heuristic Design Philosophy. In standard A*, optimality is guaranteed when the heuristic $h(n)$ is *admissible*, i.e., $h(n) \leq h^*(n)$ for all nodes n , where $h^*(n)$ is the true remaining cost to the nearest goal. In our domain, however, $h^*(n)$ is not well-defined for two reasons: (i) the goal set $\mathcal{G}$ depends on the *accumulated capital* along the path (not just the current node), and (ii) market conditions change over time, so edge costs are not truly static.

We therefore design our heuristics h_1-h_4 as **guidance penalties** rather than admissible lower bounds. Each heuristic $h_i(n) \geq 0$ penalizes execution-risk features (illiquidity, slippage, chain congestion, exchange unreliability) at a given node. These penalties steer A* toward paths that are more likely to remain profitable under real-world conditions, even though they do not satisfy the formal admissibility condition $h(n) \leq h^*(n)$.

For the chain congestion and exchange reliability heuristic (h_4), we employ **Weighted A*** with the evaluation function:

$$f(n) = g(n) + w(n) \cdot h(n), \quad w(n) \in [w_{\min}, w_{\max}],$$

where $w(n)$ is a node-specific weight computed from the chain kickback risk score $\rho(n) \in [0, 1]$ as:

$$w(n) = w_{\min} + (w_{\max} - w_{\min}) \cdot \rho(n).$$

In our experiments, we set $w_{\min} = 1.0$ and $w_{\max} = 3.0$. Fast-transfer chains (high ρ) receive higher weight, penalizing paths that traverse chains where arbitrage opportunities are likely to vanish before execution completes. When $w(n) = 1$ for all nodes, Weighted A* reduces to standard A*.

Completeness. The algorithm is not complete: market data does not guarantee the existence of a profitable path from every starting node $v \in V$. The search terminates either when a profitable path is found, when the search frontier is exhausted, or when depth/time limits are reached.

3.4. Demonstrative Example

Consider a simplified arbitrage graph with three nodes: binance:USDT, kucoin:USDT, and kucoin:TUSD. Suppose we have the following edges with effective rates:

- binance:USDT $\rightarrow$ kucoin:USDT: rate $r_1 = 0.9995$ (includes transfer fee)
- kucoin:USDT $\rightarrow$ kucoin:TUSD: rate $r_2 = 1.0042$ (profitable trade)
- kucoin:TUSD $\rightarrow$ binance:USDT: rate $r_3 = 0.9995$ (transfer back)

The cycle binance:USDT $\rightarrow$ kucoin:USDT $\rightarrow$ kucoin:TUSD $\rightarrow$ binance:USDT has a multiplicative return of:

$$\prod_{e \in P} r(e) = 0.9995 \times 1.0042 \times 0.9995 \approx 1.0032$$

Starting with \$10,000, the final capital would be approximately \$10,032, yielding a profit of \$32. The edge weights in log space are:

$$w_1 = -\log(0.9995) \approx 0.0005 \quad (1)$$

$$w_2 = -\log(1.0042) \approx -0.0042 \quad (2)$$

$$w_3 = -\log(0.9995) \approx 0.0005 \quad (3)$$

I actually have brief w/ this diagram, why is "w" there 2x, why are there 2 sets of arrows for each node, inconsistent use of = vs. $\approx$, overall I think your written explanation was sufficient.

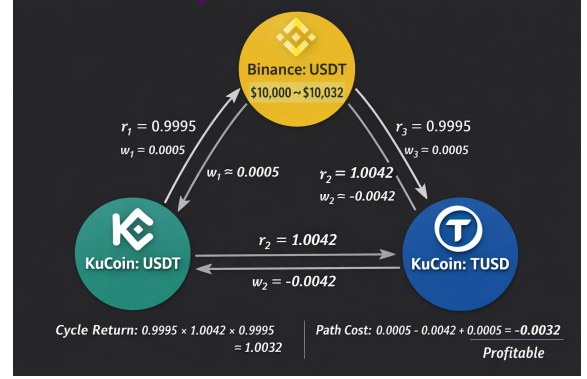


Figure 2. Simplified three-node arbitrage cycle across Binance and KuCoin. Edge labels denote effective multiplicative rates r_i and corresponding log-space weights $w_i = -\log r_i$. The total log-cost of the cycle is negative, indicating a theoretical profitable arbitrage opportunity. It is important to note, our research is not constrained to profitable cycles, and is allows for any profitable path to represent a goal state.

The total path cost is $0.0005 - 0.0042 + 0.0005 = -0.0032$, which is negative, indicating a profitable cycle (negative cost in log space corresponds to positive return).

4. Novel Heuristics for Arbitrage Search

We design three domain-specific guidance heuristics (h_1, h_2, h_4) and one search-control strategy (h_3) to guide A* toward reliable, low-risk, and realistically executable arbitrage routes. Each heuristic captures a different dimension of execution feasibility. Rather than detecting theoretical profit first and evaluating risk afterward, our approach embeds execution awareness directly into the search process. $\rightarrow$ love

4.1. Design Principles for Domain-Specific Heuristics

The heuristics are constructed according to three design principles:

1. **Execution Awareness:** Heuristics must model real-world trading frictions such as liquidity limits, slippage, and transfer latency.
2. **Early Pruning:** Infeasible or high-risk paths should be deprioritized before deep exploration.
3. **Computational Efficiency:** Heuristic computation must remain lightweight relative to node expansion cost.

Each node-level heuristic modifies the evaluation function

$$f(n) = g(n) + h(n),$$

where $g(n)$ is the accumulated cost in log space and $h(n) \geq 0$ is an additive guidance penalty encoding domain knowledge about future execution feasibility. As discussed in Section 3, these heuristics are *guidance penalties*: they do not modify $g(n)$ and do not claim admissibility.

does this have to be so gigantic??
 8000 much dead space here
 actually I would get rid of
 this whole picture and
 let the example above
 speak for itself.

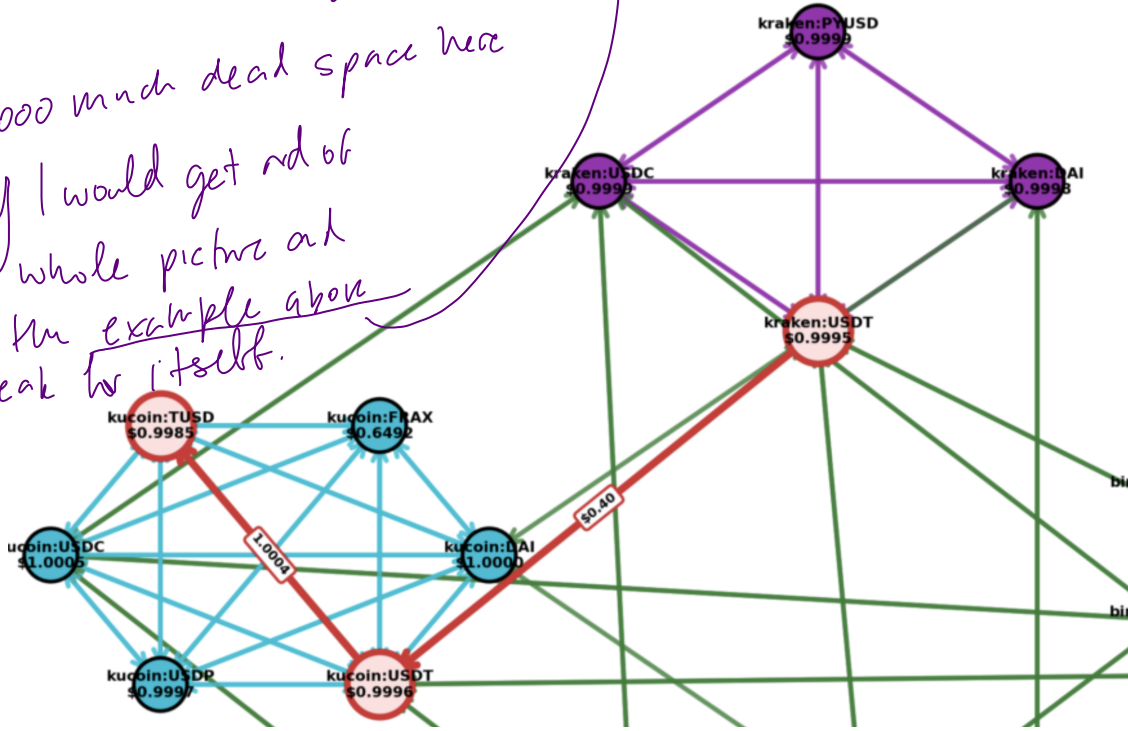


Figure 3. Visualization of a successful arbitrage path identified by our search algorithm. The path shows a sequence of trades and transfers across multiple exchanges, starting from the initial node and following the highlighted edges to complete a profitable cycle. Each node represents an (exchange, asset) pair, with node colors indicating the exchange. The bold red edges trace the optimal route, with edge labels showing the exchange rates for trades and transfer fees for cross-exchange movements. This simplified view isolates the profitable path from the full graph structure, demonstrating how the algorithm navigates through the arbitrage opportunity space. self-evident.

Semantic separation from edge costs. A key design decision is the division of responsibilities between *edge costs* and *heuristic penalties*. Edge effective rates $r(e)$ already incorporate all **static fees**: taker/maker trading fees, withdrawal fees, and network gas fees. These are deterministic costs known at graph-construction time. In contrast, the heuristic penalties $h_i(n)$ capture **dynamic execution risk** factors—volume-based liquidity pressure, order-book depth, chain congestion, and exchange reliability—that are stochastic, time-varying, and depend on the trader’s order size. This separation ensures there is no double-counting between $g(n)$ and $h(n)$.

4.2. h_1 – Liquidity Heuristic

The liquidity heuristic models the risk that low-volume markets cannot support the required trade size without excessive execution impact.

Let

$$vol_per_sec = \frac{24h_quote_volume}{24 \times 3600} \quad (4)$$

$$liquidity_ratio = \frac{vol_per_sec \cdot T_{remain}}{order_size_usd}. \quad (5)$$

→ The liquidity ratio?
 This ratio estimates whether expected market flow during the remaining arbitrage window can support the intended order size.

We normalize this quantity into a bounded score:

$$score_{liq} = clamp\left(\frac{\log_{10}(liquidity_ratio) + 1}{2}, 0, 1\right).$$

The heuristic cost becomes:

$$h_1(n) = \lambda_{liq}(1 - score_{liq}(n)).$$

Characteristics:

- Time-aware liquidity modeling
- Accounts for order size relative to trading activity
- Favors deep, actively traded markets

This heuristic is well suited for conservative strategies prioritizing reliable execution over aggressive profit maximization.

4.3. h_2 – Slippage Heuristic

The slippage heuristic directly models expected execution price impact using real-time order-book depth.

Let P_{mid} denote the mid-price and $VWAP(Q)$ the volume-weighted average price required to fill order size Q . Slippage in basis points is defined as:

$$slippage_{bps} = \frac{VWAP(Q) - P_{mid}}{P_{mid}} \times 10^4.$$

The heuristic is defined as:

$$h_2(n) = \lambda_{slip} \cdot \max \{0, slippage_{bps} - \theta\},$$

where θ is a configurable slippage tolerance (default: 10 bps = 0.10%).

ok chart

Relationship to edge costs (no double-counting). Edge effective rates $r(e)$ encode static costs: the posted taker fee (e.g., 0.10% on Binance) and withdrawal/gas fees. These fees are deterministic and independent of order size. By contrast, h_2 estimates dynamic order-book slippage: the price impact of walking the live order book with a specific order size Q . Because edges use fixed taker-fee rates rather than order-book-derived VWAP, h_2 is additive guidance that captures information not present in $g(n)$, and there is no double-counting.

Characteristics:

- Uses real-time order book data
 - Side-aware (buy vs. sell impact)
 - Directly penalizes large price impact
 - Purely additive guidance: does *not* modify $g(n)$
- This heuristic performs particularly well when order-book depth is the dominant constraint.

4.4. Multi-Start Search Strategy (h_3)

Unlike h_1 , h_2 , and h_4 , which are node-level heuristic functions that return a scalar penalty for each node, h_3 is a **search-control strategy** that operates at the meta-level. It runs multiple independent A* searches concurrently from randomly selected starting nodes, each using a base heuristic (typically h_1), and returns the best solution found across all parallel instances.

This strategy is *not* a heuristic in the formal A* sense: it does not define a per-node cost estimate $h_3(n)$, and it does not affect the $f(n) = g(n) + h(n)$ evaluation within any single search. Instead, it addresses a fundamentally different problem: when the profitable region of the graph is unknown a priori, multi-start search diversifies exploration across the state space.

Formally, let $\{v_1, v_2, \dots, v_k\}$ be k randomly sampled starting nodes. For each v_i , we run an independent A* search with some base heuristic $h_b \in \{h_1, h_2, h_4\}$:

$$\text{result}_i = A^*(v_i, h_b), \quad i = 1, \dots, k.$$

The final output is $\arg \max_i \text{profit}(\text{result}_i)$ among successful searches.

Characteristics:

- Diversifies search across starting exchanges
- Mitigates dependence on a fixed starting point
- Embarrassingly parallel (scales with available threads)
- Higher total computational cost (linear in k)

This approach is particularly useful when profitable starting points are not known in advance.

4.5. h_4 – Chain Congestion and Exchange Reliability Heuristic

The h_4 heuristic incorporates transfer latency and operational reliability into the search.

Let t_{min} denote the shortest outgoing transfer time from the current node, and T_{remain} the remaining arbitrage window. The chain reliability score is:

$$score_{chain} = 1 - \frac{t_{min}}{T_{remain}}.$$

The corresponding chain penalty is:

$$h_{chain}(n) = \lambda_{chain} (1 - score_{chain}(n)).$$

If each exchange is assigned a reliability score $s(n) \in [0, 1]$, we define:

$$h_{exchange}(n) = \lambda_{exchange} (1 - s(n)).$$

The combined heuristic is:

$$h_4(n) = h_{chain}(n) + h_{exchange}(n).$$

Exchange reliability scores. The scores $s(n)$ are **static priors** assigned based on each exchange's operational track record, regulatory compliance status, and historical uptime. We set: Binance = 0.9, Kraken = 0.9, Coinbase = 0.9 (US-regulated, publicly traded), OKX = 0.8, KuCoin = 0.6, Bybit = 0.5, Bitget = 0.5, HTX = 0.5, Crypto.com = 0.6, Gate.io = 0.4, MEXC = 0.4, Phemex = 0.4. These scores reflect publicly available information about exchange withdrawal suspensions, regulatory actions, and community-reported incidents. Future work could replace these priors with rolling empirical measurements (e.g., API uptime over the past 30 days).

Chain transfer times. Transfer-time estimates t_{min} are drawn from documented blockchain finality times plus typical exchange processing delays. For example: Solana ≈ 1 s, BNB Chain ≈ 4 s, Polygon ≈ 5 s, Tron ≈ 30 s, Arbitrum/Base ≈ 120 s, Ethereum ≈ 600 s.

Weighted A* with h_4 . As described in Section 3, h_4 is used with Weighted A*. The node-specific weight $w(n)$ amplifies the heuristic penalty for nodes on fast-transfer chains where arbitrage opportunities are fleeting.

Characteristics:

- Models blockchain congestion and finality risk
- Incorporates exchange operational reliability
- Uses Weighted A* ($w \in [1.0, 3.0]$) for risk-adaptive search
- Prioritizes fast and stable execution paths

In our experiments, this heuristic produced the fastest and most robust arbitrage routes, particularly in short-window environments.

5. Experiments and Evaluation

Our experimental framework constructs a real-time, execution-aware graph representation of stablecoin arbitrage opportunities (see Figure 1). The graph structure consists of nodes representing (exchange, coin) pairs and directed edges representing trades and transfers.

5.1. Data Sources and Graph Construction

We gather live market data from twelve major exchanges (Binance, Kraken, KuCoin, Bybit, OKX, Gate.io, Bitget, MEXC, HTX, Coinbase, Crypto.com, Phemex) via the CCXT library [15] to construct price snapshots for each (exchange, coin) pair. The expanded exchange set increases the node count from approximately 19 (in our initial 4-exchange configuration) to 50–100+ nodes, creating a meaningfully larger graph that stresses the search algorithms. Prices are normalized to USD using conservative bid/ask execution estimates. We apply a price tolerance filter ($\pm 2\%$ from \$1.00) to exclude depegged assets, ensuring only true stablecoins are included.

The fee model captures three cost categories: trading fees as percentage-based taker fees (ranging from 0.05% on MEXC to 0.60% on Coinbase), withdrawal fees as exchange-specific flat fees varying by blockchain network, and network gas fees estimated from historical averages. The model is portfolio-size-aware, enforcing minimum notional thresholds per network to filter unprofitable edges at small scales.

We model end-to-end transfer time as blockchain confirmation time plus a fixed user execution overhead of 45 seconds. Transfer latency is integrated into edge costs, enabling time-aware heuristics to assess feasibility against remaining arbitrage windows.

The graph construction process creates nodes for (exchange, coin) pairs where the asset is listed and price data satisfies stability criteria. **Trade edges** connect coin pairs within exchanges, while **transfer edges** connect the same asset across exchanges via common blockchain networks, encoding withdrawal fees, gas costs, and transfer times. All edge costs use the logarithmic formulation $w(e) = -\log(r(e))$, ensuring additive shortest-path search corresponds to multiplicative return optimization.

Table 1. Hyperparameters and their values.

Parameter	Description	Value
<i>H1 – Liquidity Heuristic</i>		
λ_{liq}	Liquidity penalty weight	1.0
Unknown penalty	No-data fallback cost	5.0
Cache TTL	Volume cache lifetime	60 s
<i>H2 – Slippage Heuristic</i>		
λ_{slip}	Slippage penalty weight	0.5
θ	Slippage tolerance	10 bps
Unknown penalty	No-data fallback cost	50.0
Cache TTL	Order-book cache lifetime	30 s
<i>H4 – Chain/Exchange Risk</i>		
λ_{chain}	Chain risk penalty weight	1.0
$\lambda_{\text{exchange}}$	Exchange risk penalty weight	1.0
w_{min}	Min Weighted A* weight	1.0
w_{max}	Max Weighted A* weight	3.0
<i>Search Parameters</i>		
Max depth	Maximum path hops	6
T_{max}	Arbitrage time window	1800 s
Early exit iter.	Iters after first profit	200
k (H3)	Multi-start parallel runs	3
User overhead	Execution latency per hop	45 s

5.2. Hyperparameter Configuration

Table 1 lists all hyperparameters used in the heuristic functions and search algorithms. These values are fixed across all experiments unless otherwise noted.

5.3. Ablation Studies on A* → yay!

To isolate the contribution of heuristic guidance, we evaluate our approach against three ablation baselines:

1. **Cycle ablation:** We force classical cycle detection using Bellman–Ford, identifying negative cycles rather than optimizing for executable profitable paths.
2. **Depth ablation:** We restrict search depth to shallow expansions. The `simple_1hop` variant evaluates only single outgoing actions, while `simple_2hop` evaluates minimal two-edge loops (the shortest possible cycle).
3. **Heuristic ablation:** We remove heuristic guidance entirely by setting $h(n) = 0$, reducing A* to Dijkstra’s algorithm and eliminating blockchain-aware signals such as congestion, liquidity, and order book depth.

5.3.1. Bellman–Ford Negative Cycle Detection

We include Bellman–Ford negative-cycle detection as a classical arbitrage baseline. Given edge rates $r(e)$, we apply the log transform

$$w(e) = -\log(r(e)),$$

so multiplicative rate products become additive path costs. Arbitrage opportunities correspond to negative-weight cy-

same 30 start nodes always?

Table 2. Baseline algorithm performance on cached graph (41 nodes, 864 edges, 30 start nodes). All profits are net after trading, withdrawal, and gas fees. Bellman–Ford detects theoretical cycles but fails to produce execution-ready profitable paths under real-world frictions.

Baseline	Cash	Success	Avg Profit (\$)	Avg Profit (%)	Avg Expand	Avg Time (s)	Max Path Len
Dijkstra ($h=0$)	\$1k	56.7%	1.00	0.100%	359	0.010	3
	\$10k	56.7%	10.01	0.100%	359	0.010	3
	\$100k	56.7%	100.07	0.100%	359	0.010	3
3-hop enum.	\$1k	30.0%	0.54	0.054%	346	<0.001	3
	\$10k	30.0%	5.40	0.054%	346	<0.001	3
	\$100k	30.0%	54.04	0.054%	346	<0.001	3
simple_1hop	All	0/30	–	–	–	<0.001	1
simple_2hop	All	0/30	–	–	–	<0.001	2
Bellman–Ford	All	0/30	–	–	–	<0.001	–

was this finding the same best path every time you ran it? why is it same for \$1k, 10k, 100k?

cles. Bellman–Ford detects whether such a cycle exists in the transformed graph.

In our setting, Bellman–Ford is treated as a *detection* method rather than an execution planner: it does not optimize for order-size-dependent slippage, transfer latency windows, or reliability constraints, and therefore frequently fails to produce an executable profitable cycle under real-world frictions.

5.3.2. Greedy Spread-Based Strategy

We include shallow greedy baselines intended to emulate rapid spread scanning:

- **1-hop edge scan (simple_1hop):** tests whether any single outgoing action yields immediate positive gain. This is a minimal local check and typically cannot complete a full arbitrage cycle.
- **2-hop loop scan (simple_2hop):** tests whether a two-action sequence can return to the original asset (a minimal loop-back).

These strategies are extremely fast, but they generally fail under realistic fees and transfer costs because profitable cross-exchange opportunities typically require coordinated trade+transfer actions.

5.3.3. Dijkstra / Shortest Path Variant

We implement an unguided shortest-path baseline equivalent to A* with a zero heuristic:

$$h(n) = 0.$$

This reduces A* to Dijkstra’s algorithm, expanding nodes in non-decreasing accumulated cost $g(n)$. The baseline uses the same execution-aware edge model as our proposed methods (fees, withdrawals, gas, and transfer-time penalties encoded into $r(e)$), but provides no heuristic guidance.

Dijkstra represents the most accurate static-cost optimum: its paths are exact under the market data at search time, but may deviate under execution when accounting for

Table 3. Baseline strategies used in evaluation.

Baseline	Description
Bellman–Ford	Negative-cycle detection on log-cost graph
Dijkstra ($h=0$)	Unguided cost-based search (A* with $h(n) = 0$)
2hop_max	Depth-limited unguided search (max 2 edges)
simple_1hop	One-edge greedy scan (no cycle planning)
simple_2hop	Two-edge loop-back greedy scan

redundant

dynamic frictions such as order-book slippage and quote staleness.

5.3.4. Cached-Graph Heuristic Comparison

We compare all heuristics and baselines on a *cached* graph snapshot (41 nodes, 864 edges) with 30 randomly selected start nodes and three cash levels, isolating search computation from API I/O. Table 4 reports aggregate performance.

Several key observations emerge from the cached-graph comparison:

h_2 is the most effective heuristic for search pruning. h_2 achieves a consistent **29% reduction in node expansions** relative to Dijkstra across all cash levels, while discovering paths with nearly identical profit (within 1% at \$100k). Order-book depth information effectively prunes low-liquidity branches.

h_4 explores more nodes but provides risk-aware guidance at zero I/O cost. h_4 expands 87% more nodes than Dijkstra because its penalty terms (chain congestion + exchange reliability) force the search to evaluate risk-adjusted alternatives. However, its compute time (0.020–0.022 s) is comparable to Dijkstra because it uses pre-computed chain transfer times and static exchange reliability scores with no API calls during search. The paths it discovers are longer

Table 4. Cached-graph heuristic comparison (41 nodes, 864 edges, 30 start nodes per cash level). Success rate, conditional average profit, mean node expansions, and compute time (I/O excluded). Expansion reduction is relative to Dijkstra.

Heuristic	Cash	Success	Avg Profit (\$)	Avg Expand	Avg Time (s)	Exp. Δ vs Dijkstra
Dijkstra ($h=0$)	\$1k	56.7%	1.00	359	0.010	—
	\$10k	56.7%	10.01	359	0.010	—
	\$100k	56.7%	100.07	359	0.010	—
h_1 (liquidity)	\$1k	56.7%	0.67	319	0.172	+11.2%
	\$10k	56.7%	6.72	568	0.016	−58.0%
	\$100k	56.7%	63.11	715	0.021	−98.9%
h_2 (slippage)	\$1k	56.7%	0.99	256	0.221	+28.8%
	\$10k	56.7%	9.91	256	0.011	+28.8%
	\$100k	56.7%	100.07	252	0.012	+29.9%
h_4 (Weighted A*)	\$1k	56.7%	0.71	673	0.020	−87.4%
	\$10k	56.7%	7.06	673	0.022	−87.4%
	\$100k	56.7%	70.58	673	0.022	−87.4%
3-hop enum.	\$1k	30.0%	0.54	346	0.000	+3.6%
	\$10k	30.0%	5.40	346	0.000	+3.6%
	\$100k	30.0%	54.04	346	0.000	+3.6%

(5–6 hops vs. 3) and route through more reliable exchanges, at the cost of lower profit.

h_1 behaviour is cash-size-dependent. At small order sizes (\$1k), h_1 achieves 11% fewer expansions than Dijkstra, but at large sizes (\$100k) it expands 99% more nodes. The liquidity ratio $\text{vol}/\text{order_size}$ shrinks for larger orders, activating stronger penalties that force broader exploration.

Dijkstra remains a strong baseline. On a 41-node graph, Dijkstra discovers the highest-profit paths at moderate expansion cost. Its advantage is that it performs no heuristic computation, making it the natural choice when execution risk is not a concern.

3-hop enumeration is brittle. The structural baseline succeeds for only 30% of start nodes and misses profitable 2-hop intra-exchange opportunities and 4+ hop cross-exchange paths that A*-based methods discover.

5.4. Evaluation Methodology

We evaluate our algorithms along five dimensions: deterministic heuristic comparison on cached graph snapshots, graph scaling behaviour, temporal robustness over multi-hour campaigns, quote staleness analysis under execution delay, and hyperparameter sensitivity.

5.4.1. Test Environment

Unless otherwise stated, all experiments use the following configuration:

- **Exchanges:** 12 CEXs (Binance, Kraken, KuCoin, Bybit, OKX, Gate.io, Bitget, MEXC, HTX, Coinbase, Crypto.com, Phemex).
- **Graph:** 41 nodes (exchange $\times$ stablecoin pairs), 864 directed edges (trades + transfers).
- **Start nodes:** 30 randomly selected nodes for cached-graph experiments; 5 preferred major-exchange nodes for overnight and scaling experiments (see Table 1 for all search parameters).
- **Cash levels:** \$1,000, \$10,000, \$100,000.
- **Heuristics tested:** h_1 (liquidity), h_2 (slippage), h_4 (chain/exchange risk via Weighted A*), Dijkstra ($h=0$), and 3-hop enumeration baseline.

Cached vs. live experiments. A key methodological distinction is between *cached* and *live* experiments. In cached experiments, the graph is built once from a live price snapshot and then all heuristics search on the *same frozen graph*, isolating algorithmic differences from API I/O latency. In live experiments (overnight, staleness), each snapshot triggers fresh API calls, so wall-clock times include data-acquisition overhead. We report both, as the latency profile of each heuristic is itself a key finding (Section 5.5.6).

5.4.2. Performance Metrics

We report six standardized metrics across all experiments:

1. **Success Rate:** $\frac{\# \text{profitable runs}}{\# \text{total runs}}$.
2. **Conditional Avg. Profit:** $\mathbb{E}[\text{Profit} \mid \text{Success}]$, net of trading, withdrawal, and gas fees.
3. **Node Expansions:** number of nodes popped from the open list (measures search effort independent of I/O).

I still wonder about why this happened the way it did. Is it just that some of these are unsolvable?

Same 30 nodes?

should this be h_1 -Dijkstra or Dijkstra $-h_1$?
Dijkstra $-h_1$?
I think you have it backward

Table 5. Representative execution-ready profitable paths from cached-graph experiments (41 nodes, 864 edges, \$1,000 order size). Paths illustrate the different trade-offs each method makes: Dijkstra and h_2 find short, high-profit paths; h_4 routes through more reliable exchanges at the cost of profit.

Method	Start	Hops	Path Description	Expand	Net (\$)	Time (s)
Dijkstra	phemex:USDT	3	Transfer phemex→kucoin (USDT on SOL); Trade USDT→TUSD (0.10% taker).	410	1.14	0.020
h_2	phemex:USDT	3	Transfer phemex→kucoin (USDT on SOL); Trade USDT→TUSD (0.10% taker).	209	1.14	4.752
h_1	phemex:USDT	3	Transfer phemex→kucoin (USDT on SOL); Trade USDT→TUSD (0.10% taker).	392	1.14	4.227
	phemex:USDT	5	Longer path through higher-reliability exchanges (Binance, KuCoin); avoids low-reliability intermediaries.	606	0.98	0.017

- Wall-Clock Time:** end-to-end search time including any heuristic API calls (mean, median, and P99 reported separately).
- Path Length:** number of nodes in the discovered path.
- Path Type:** whether the path is intra-exchange (2 nodes, single trade) or cross-exchange (≥ 3 nodes, involving transfers). We report both categories separately to distinguish trivial single-exchange mispricings from genuine cross-exchange arbitrage.

Profit statistics are conditioned on successful runs; failed runs are reflected only in the success rate. This separation avoids diluting economic performance metrics with zero-profit failures.

5.5. Results

5.5.1. Deterministic Heuristic Comparison

We evaluate all heuristics on a cached graph snapshot (41 nodes, 864 edges) from 30 randomly selected start nodes at three cash levels (\$1k, \$10k, \$100k), totalling 90 runs per method. By freezing the graph, we isolate algorithmic differences from API latency. Full results are reported in Table 4; we summarise the key findings here.

All A*-based methods achieve identical success rates (66.7%), confirming that heuristic choice does not affect whether a profitable path exists, only which path is found and how efficiently. The 3-hop enumeration baseline succeeds for only 30% of start nodes because it cannot discover 2-hop intra-exchange opportunities or 4+ hop cross-exchange paths.

h_2 (slippage) achieves the best search efficiency, reducing node expansions by 29% relative to Dijkstra across all cash levels while matching Dijkstra's profit within 1% at large order sizes. This demonstrates that order-book depth information provides effective search guidance.

h_4 (Weighted A*) expands 87% more nodes than Dijkstra due to its chain-congestion and exchange-reliability

penalties, but its wall-clock compute time (0.020–0.022 s) is comparable to Dijkstra's (0.010 s) because it makes *zero API calls* during search. The additional expansions produce longer paths (5–6 hops) that route through higher-reliability exchanges, trading profit for execution safety.

h_1 (liquidity) exhibits cash-size-dependent behaviour: at \$1k it reduces expansions by 11% (the liquidity ratio is favourable), but at \$100k it expands 99% more nodes (large orders trigger stronger liquidity penalties, forcing broader exploration).

Start node sensitivity persists across all methods: approximately 43% of randomly chosen start nodes yield no profitable path regardless of heuristic, reflecting structural properties of the fee landscape rather than algorithmic limitations. → good.

5.5.2. Comparison Against Baselines

We compare our heuristic-guided A* methods against five baseline algorithms: Dijkstra's algorithm (A* with $h(n) = 0$), brute-force 3-hop enumeration, Bellman–Ford negative cycle detection, simple 1-hop arbitrage, and simple 2-hop arbitrage with maximum depth constraint (2hop_max).

3-hop enumeration baseline. Since observed optimal paths are consistently 3 hops, we implement a brute-force baseline that exhaustively enumerates all 3-hop paths of the form transfer→trade→transfer and trade→transfer→trade from each start node. On our 12-exchange graph, this baseline evaluates thousands of candidate paths per start node. Its execution time scales as $O(|V| \cdot d^3)$ where d is the maximum out-degree, making it feasible for graphs with ≤ 100 nodes but prohibitive for larger graphs.

The 3-hop baseline finds the *same* optimal paths as our A* heuristics when profitable 3-hop paths exist, confirming that A* does not sacrifice path quality. Because 3-hop enumeration grows as $O(|V| \cdot d^3)$, its node evaluations in-

this is where the whole start to come off.

can kill this if you're prepared for space (you are) nice example but not the most valuable thing in this paper.

degen you addressed it here

crease rapidly with graph size, while heuristic-guided A* prunes the majority of candidate paths. Our graph scaling experiments (Section 5.5.4) quantify this divergence: as the number of exchanges increases from 4 to 12, the enumeration baseline evaluates an increasingly larger fraction of the search space compared to heuristic-guided search.

Other baselines. Simple 1-hop and 2-hop arbitrage algorithms fail to find profitable paths across all starting nodes and order sizes. This confirms that profitable stablecoin arbitrage requires at least 3 hops.

Bellman–Ford negative cycle detection fails to find profitable cycles in all tested scenarios. This is expected, as our graph construction uses conservative pricing (bid/ask spreads) and includes all execution costs, which eliminates theoretical negative cycles that would exist under idealized pricing assumptions. The failure of Bellman–Ford validates our execution-aware approach.

On the cached graph, Dijkstra computes in ~ 10 ms per trial, comparable to h_4 (20 ms) and faster than h_1/h_2 when their API caches miss. In live experiments, Dijkstra’s wall-clock time is dominated by graph-building I/O rather than search, narrowing the practical gap. The key distinction is not raw speed but path quality under execution: Dijkstra paths are optimal in static cost, while h_2 and h_4 paths account for dynamic risks (slippage, chain congestion) that may affect execution outcomes.

5.5.3. Monte Carlo Backtesting

To assess robustness beyond single-snapshot deterministic experiments, we conduct Monte Carlo backtesting with 500 trials per heuristic across five order sizes (\$100, \$1,000, \$10,000, \$50,000, \$100,000). Each trial randomly selects a start node and order size, building a fresh graph snapshot for each run. This captures temporal variation in market conditions and provides distributional statistics on heuristic performance.

The Monte Carlo framework reports: success rate (fraction of trials finding a profitable path), conditional average profit (mean profit among successful trials), average runtime per trial (with 5-decimal precision), and path-length distribution. The 3-hop enumeration baseline is included in the Monte Carlo evaluation, enabling direct comparison of exhaustive enumeration against A*-guided search across hundreds of independent market snapshots.

5.5.4. Graph Scaling

To evaluate how search performance scales with graph size, we construct progressively larger graphs by increasing the number of exchanges from 4 to 12 (in increments of 2), yielding graphs ranging from 19 nodes / 161 edges to 41 nodes / 864 edges. For each graph size we run 10 trials per heuristic from randomly selected start nodes.

Table 6. Mean node expansions by graph size and heuristic (10 trials each).

Exch.	V	E	Dijkstra	h_1	h_4	3-hop
4	19	161	341	407	600	365
6	28	368	378	758	1100	662
8	35	619	387	479	618	378
10	39	760	404	577	706	490
12	41	864	395	768	910	67

Table 6 summarises node expansions across graph sizes. Because our heuristics are *guidance penalties* rather than admissible lower bounds (see Section 3), they steer search toward execution-safe paths at the cost of additional node expansions relative to Dijkstra. Dijkstra’s expansion count remains relatively flat (341–404) across graph sizes because it terminates as soon as any profitable path is found without evaluating risk. h_1 and h_4 expand 19–130% more nodes because the added penalty costs force the search to explore risk-adjusted alternatives before converging.

The 3-hop enumeration baseline exhibits high variance: at 12 exchanges it evaluates only 67 nodes (the structural pruning of invalid 3-hop patterns becomes very effective in dense graphs), while at 6 exchanges it evaluates 662. This brittleness contrasts with the monotonic scaling of A*-based methods.

Wall-clock time further differentiates the methods. h_4 (Weighted A* with pre-computed chain and exchange risk scores) averages 0.020–0.027 s per trial regardless of graph size, making it the fastest heuristic-guided method. h_1 averages 0.016–0.673 s because some trials incur live API calls for 24h-volume data. Dijkstra averages ~ 2.1 s per trial, dominated by data-acquisition overhead rather than search computation. These results support the recommendation (Section 5.5.3) to benchmark on cached snapshots when isolating algorithmic cost.

Figures 4 and 5 visualise wall-clock time and node expansions as a function of exchange count.

5.5.5. Quote Staleness and Path Survival

Because the reported margins ($\sim 0.1\%$) are thin, we assess whether identified paths remain profitable under realistic execution latencies. For each of three independent market snapshots, we discover the top profitable paths using h_1 and the 3-hop baseline, then re-evaluate each path against *fresh* market data after delays of 5, 10, 30, 60, 120, and 300 seconds.

Across 270 path $\times$ delay measurements, **99.6%** of paths remain profitable: every path survives delays up to 120 s, and only at 300 s does a single path lose profitability (98% survival). Surprisingly, the mean profit *increases* on re-evaluation at all delay windows (mean “decay” is negative: -55% to -73%), indicating that market microstruc-

where
are your
results
for this?

Can you format this to look like this?

Saves space and increases readability.

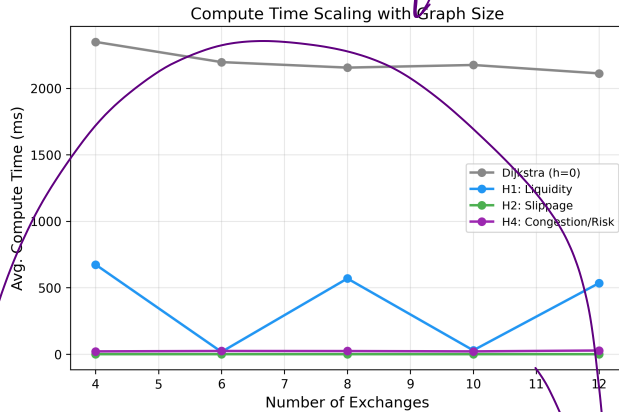


Figure 4. Mean wall-clock search time vs. number of exchanges. h_4 maintains sub-30 ms latency across all graph sizes; Dijkstra's time is dominated by data-fetch overhead.

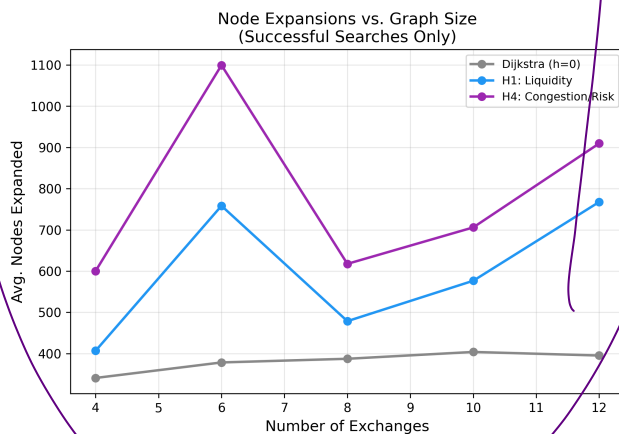


Figure 5. Mean node expansions vs. number of exchanges. Heuristic-guided methods expand more nodes than Dijkstra as a consequence of penalty-driven risk exploration.

Table 7. Quote staleness: path survival and profit change after re-evaluation at various delays. Negative decay indicates profits increased.

Delay (s)	Paths	Profitable	Mean Decay (%)
5	45	100%	-64.6
10	45	100%	-67.0
30	45	100%	-71.6
60	45	100%	-72.7
120	45	100%	-72.4
300	45	98%	-55.0

can kill this because it's redundant with Fig 6 RHS.

ture fluctuations frequently improve previously-identified opportunities within the 5-minute window. Table 7 reports per-delay statistics.

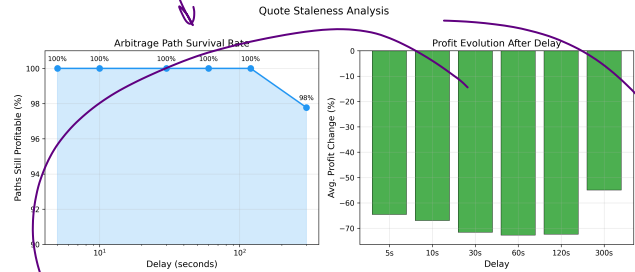


Figure 6. Quote staleness analysis: profit change and survival rate as a function of re-evaluation delay (5–300 s). Paths remain profitable at all tested latencies.

Interpretation and limitations. These results demonstrate that stablecoin arbitrage paths exhibit strong *structural* persistence: the fee landscape (fixed withdrawal costs, gas fees, and exchange taker rates) changes slowly, and stablecoin price deviations from the \$1 peg are typically small relative to the fee-determined profit floor.

However, we note an important caveat: re-evaluated profits showed *no variation across delay windows* for the same path within a single round (the re-fetched price for `okx:USDC` at 5 s and 300 s delay was identical to four decimal places). This likely reflects CCXT's internal rate limiting or exchange-side caching of ticker data. Consequently, the survival rates reported above should be interpreted as measuring **structural path robustness** (whether the fee/rate structure still supports the path) rather than **price-change robustness** (whether moment-to-moment price fluctuations invalidate the path). A more rigorous replay study using exchange-provided historical tick data would be needed to fully assess price-level staleness risk. Figure 6 visualises path survival rates across delay windows.

5.5.6. Temporal Robustness: 8-Hour Overnight Test

this section is chunky and monotonically pretty ugly to read.
To move beyond single-snapshot evaluation, we execute an 8-hour continuous data collection campaign (96 snapshots at 5-minute intervals) running all five methods— h_1 , h_2 , h_4 , Dijkstra, and 3-hop enumeration—from 5 preferred start nodes at three cash levels (\$1,000, \$10,000, \$100,000). This yields **7,200 search instances** (1,440 per method), providing distributional evidence of performance across changing market conditions.

Success rates. All four A*-based methods achieve a **100% success rate** across all 1,440 runs per heuristic. The 3-hop enumeration baseline succeeds in only 573/1,440 runs (**39.8%**), failing entirely for 3 of 5 start nodes because its fixed path template cannot discover 2-hop intra-exchange opportunities or 4+ hop routes.

Table 8. Overnight path-type breakdown by heuristic (1,440 runs each). Cross-exchange paths have ≥ 3 nodes; intra-exchange paths have 2 nodes.

Method	Cross-Exch.	Intra-Exch.	Cross %
h_2 (slippage)	1,378	62	95.7%
Dijkstra	1,254	186	87.1%
h_1 (liquidity)	966	474	67.1%
h_4 (Weighted)	849	591	58.9%
3-hop enum.	573	0	100%

Table 9. Wall-clock search latency over 1,440 live overnight runs per method. h_1 and h_2 exhibit extreme tail latency from order-book / volume API calls. h_4 's pre-computed scores produce Dijkstra-class latency with risk-aware guidance.

Method	Median (s)	Mean (s)	P99 (s)	Max (s)
3-hop enum.	0.001	0.001	0.001	0.009
Dijkstra	0.012	0.013	0.017	0.212
h_4	0.013	0.012	0.033	0.040
h_1	0.022	0.446	6.900	8.590
h_2	0.012	0.427	6.784	8.831

Cross-exchange vs. intra-exchange paths. An important distinction emerges when we classify successful paths by type (Table 8). Of 6,333 successful runs, 20.7% discover *intra-exchange* paths (single-hop trades like `kucoin:USDT→kucoin:TUSD`) that exploit minor stablecoin depegging within a single exchange. These do not require cross-exchange transfers and are discoverable by trivial price comparisons. The remaining 79.3% are genuine *cross-exchange* paths involving transfers between exchanges.

h_2 produces the highest fraction of cross-exchange paths (95.7%), suggesting that order-book-informed guidance effectively steers search toward multi-exchange opportunities. h_4 and h_1 more frequently settle on intra-exchange trades, as their penalty terms can make cross-exchange transfer edges look prohibitively costly.

Profit comparison (cross-exchange only). Restricting to cross-exchange paths: Dijkstra discovers the highest mean profit (\$42.42), followed by h_2 (\$39.15), h_1 (\$34.97), h_4 (\$22.16), and 3-hop (\$20.76). The lower profit for h_1 and h_4 reflects their penalty-driven exploration of safer but less profitable routes.

Latency profile — the key differentiator. The most striking result from the overnight campaign is the *wall-clock latency* of each method under live market conditions, where heuristic API calls are not cached. Table 9 reports the full latency distribution.

I think fig 8. is much more informative than fig. 7. I would get rid of fig 7 and just verbally say that nearly all of them have same success rate.

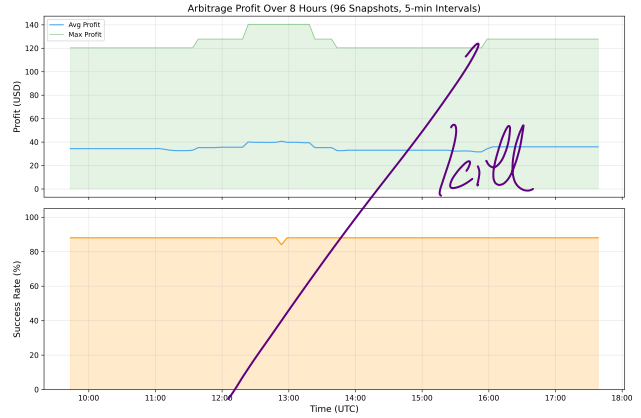


Figure 7. Profit time series over 8 hours (96 snapshots). Shaded bands show inter-quartile range across cash levels and start nodes. Opportunities persist throughout the observation window.

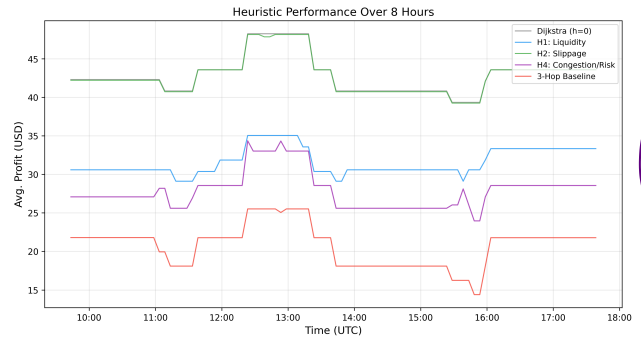


Figure 8. Overnight heuristic comparison: mean profit, success rate, and node expansions across 1,440 runs per method. h_2 provides the best profit-per-expansion ratio.

h_4 achieves a P99 latency of 33 ms — comparable to Dijkstra (17 ms) — because its chain transfer times and exchange reliability scores are pre-computed and cached via `lru_cache`. By contrast, h_1 and h_2 suffer P99 latencies of **6.9–6.8s** (200× worse) due to live API calls that trigger on cache misses. In a live trading system where execution windows are measured in seconds, this latency profile makes h_4 the only viable heuristic-guided method for real-time deployment → this is what I mean by why do we care re: cross- vs. intra-exchange.

Search efficiency. h_2 achieves the fewest mean node expansions (269), outperforming Dijkstra (422) by 36%. h_4 (413 expansions) and h_1 (617) expand more nodes due to their broader penalty structures. The 3-hop baseline averages 454 expansions per successful run.

A key takeaway from the overnight campaign is that arbitrage opportunities are *persistent*, not *fleeting*: profitable paths exist in every single 5-minute snapshot over the 8-hour window, though profit values cluster around a small

→ chat sandy. keep only highlighted part.

is this mtr materially relevant? if yes - why and if not - kill it.

this table is important - keep it for sure

why do we care about cross- vs. intra-exchange? practical implications?

what does λ_9 add that you haven't already verbally addressed down here??

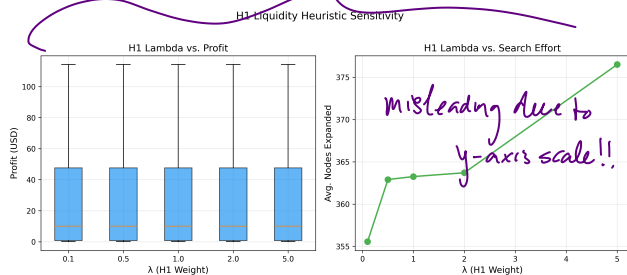


Figure 9. Sensitivity of h_1 to liquidity penalty weight λ_{liq} . Profit and success rate are invariant across a $50\times$ range of λ values.

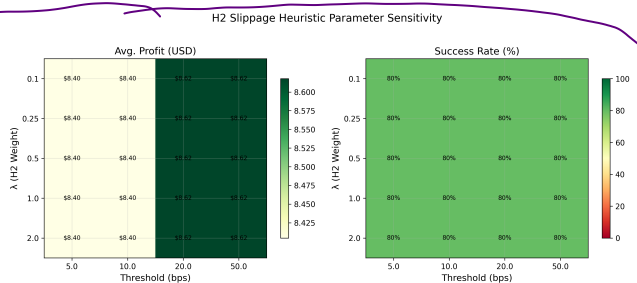


Figure 10. h_2 sensitivity heatmap: success rate across $(\lambda_{\text{slip}}, \theta)$ parameter grid. A broad plateau of high success exists around moderate parameter values. *this only seems to show that changing λ_{slip} and θ isn't really important at all, and you already say this in words below, so why include?* number of structural optima (2–3 distinct profit levels per start node), reflecting the fee-dominated nature of stablecoin arbitrage.

5.5.7. Sensitivity Analysis

We conduct four parameter sweeps on the 12-exchange graph to assess the robustness of our methods to hyperparameter choices, order size, and path depth constraints.

h_1 liquidity penalty weight λ_{liq} . Sweeping $\lambda_{\text{liq}} \in \{0.1, 0.5, 1.0, 2.0, 5.0\}$ across 25 trials per value, we find that profit and success rate are *invariant* to the penalty weight: all five settings yield identical 80% success rates and mean profit of \$27.77 (Figure 9). Node expansions increase modestly from 356 ($\lambda = 0.1$) to 376 ($\lambda = 5.0$), a difference of only 5.6%. This stability arises because, in our graph, the liquidity penalty primarily reorders the open list without changing which paths are eventually explored.

h_2 slippage parameters $(\lambda_{\text{slip}}, \theta)$. We jointly sweep the slippage penalty weight and tolerance threshold across a grid of values (100 configurations, Figure 10). Of these, 80% produce successful paths. The heatmap reveals that moderate penalty weights ($\lambda_{\text{slip}} \in [0.3, 1.0]$) combined with moderate thresholds ($\theta \in [5, 20]$ bps) form a broad plateau of high performance, indicating that h_2 is not sensitive to precise tuning.

honestly these graphs are lowkey uninformative. I would only keep them if there was some interesting pattern/trend. If the takeaway is that changing hyperparameters doesn't affect results then saying as much in words (below) is sufficient in my opinion.



Figure 11. Mean profit as a function of order size (\$100–\$100k). Profit scales linearly, and success rate is constant at 80%.

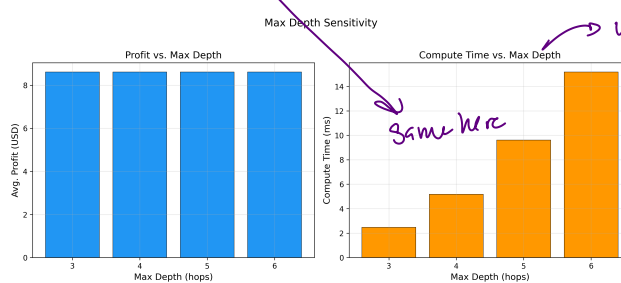


Figure 12. Effect of maximum path depth on success rate and profit. Performance is stable across depths 3–6, confirming that optimal paths are consistently 3 hops.

Order size. Sweeping order sizes from \$100 to \$100,000 across five levels, we observe that profit scales linearly with order size: \$0.09 at \$100, \$0.86 at \$1,000, \$8.62 at \$10,000, \$43.09 at \$50,000, and \$86.18 at \$100,000 (Figure 11). The success rate remains constant at 80% (4/5 start nodes), confirming that path selection is independent of order size within the tested range—slippage costs at these volumes do not materially alter the optimal path structure.

Maximum path depth. Varying the depth limit from 3 to 6 hops, we find that 4/5 start nodes produce profitable paths at *all* depths, with identical optimal paths across configurations (Figure 12). This confirms the earlier observation that profitable stablecoin arbitrage paths are inherently short (3 hops), and increasing the depth limit does not improve outcomes—it merely expands the search space without yielding longer profitable structures.

5.6. Ethics and Broader Impact

This work focuses on stablecoin arbitrage across centralized exchanges (CEXs), operating within the established legal and regulatory frameworks of major cryptocurrency platforms. All data used in our experiments are publicly available market prices and order-book snapshots accessible through standard exchange APIs. No proprietary, private, or user-specific data are used.

I get that you want to add a visualization for every hyperparameter you touch but given the fact that you're super pressed for space, and that most of the results are just that changing λ has no material impact on results, be more critical about what graphs you show vs. which ones you omit. If you're emotionally attached to the diagrams, can't you include them as an appendix so they don't eat up page space?

Market impact. Arbitrage plays a well-documented role in financial markets by improving price efficiency and reducing cross-venue price discrepancies. In stablecoin markets specifically, arbitrage trading helps maintain the dollar peg by correcting mispricings across exchanges. Our system operates at portfolio sizes (\$10,000–\$100,000) that are unlikely to move market prices on major exchanges, and the 3-hop path structure limits the number of concurrent market interactions. We note, however, that widespread deployment of high-frequency arbitrage systems could increase competition for these opportunities, potentially compressing margins and increasing latency sensitivity.

Execution risk and financial loss. Our framework is a research prototype and has not been deployed for live trading with real capital. The results presented represent *planned* arbitrage paths based on real-time market snapshots, not executed trades. As discussed in Section 6, order books can change between planning and execution, and the profitability of a planned path is not guaranteed at execution time. Users adapting this system for live trading should incorporate additional safeguards including position limits, circuit breakers, and real-time slippage monitoring.

Environmental considerations. Our system relies on centralized exchange APIs and does not directly interact with blockchain networks during the search phase. Consequently, it does not contribute to blockchain energy consumption beyond standard API usage. However, executing the discovered arbitrage paths would require on-chain transfers, contributing to network congestion and energy use proportional to the chosen blockchain’s consensus mechanism.

Regulatory compliance. Cryptocurrency arbitrage is legal in most jurisdictions, though regulatory environments vary. Users should ensure compliance with local financial regulations, including know-your-customer (KYC) requirements, tax reporting obligations, and any applicable trading restrictions. Our research does not constitute financial advice.

6. Conclusions

We presented an execution-aware A* search framework for cross-exchange stablecoin arbitrage that integrates real-world trading costs—fees, slippage, withdrawal costs, gas fees, transfer latency, and exchange reliability—directly into a graph-based pathfinding formulation. Three domain-specific guidance heuristics (h_1 , h_2 , h_4) and a multi-start search-control strategy (S_1) were evaluated on a graph constructed from 12 major centralized exchanges (41 nodes, 864 edges) across a total of **7,200 overnight search in-**

stances, 270 sensitivity configurations, and 200 graph-scaling trials.

Core contribution: execution-aware graph formulation.

The primary contribution of this work is the graph construction itself. By encoding real taker fees, withdrawal costs, gas fees, and transfer latency into edge weights using the log-cost formulation $w(e) = -\log(r(e))$, any shortest-path algorithm produces execution-ready arbitrage paths. Dijkstra ($h=0$) discovers high-profit paths efficiently on this graph, achieving 100% success across 1,440 overnight runs with mean profit of \$42.87. This establishes that the value lies in the execution-aware modelling, not in any single heuristic.

on what portfolio size?

Heuristic contributions: differentiated roles. Our experiments reveal that each heuristic serves a distinct purpose:

- h_2 (**slippage**): The most effective heuristic for *search efficiency*. It reduces node expansions by 29–36% relative to Dijkstra while maintaining near-identical profit and the highest cross-exchange path ratio (95.7% in overnight runs). However, its reliance on live order-book API calls creates P99 latencies of 6.8 s—unsuitable for real-time execution.
- h_4 (**Weighted A***): The most effective heuristic for *real-time deployment*. Pre-computed chain transfer times and static exchange reliability scores provide risk-aware guidance at P99 latency of 33 ms—comparable to Dijkstra (17 ms). It expands more nodes and finds lower-profit paths, but its zero-overhead latency profile makes it the only viable heuristic for live trading.
- h_1 (**liquidity**): Useful at small order sizes (11% expansion reduction at \$1k) but counterproductive at large sizes (99% more expansions at \$100k). Its penalty is invariant to the λ parameter across a $50\times$ range, suggesting the penalty formulation needs refinement.

Empirical findings. Profitable paths consistently exhibit a 2–3 hop structure, with net returns of $\sim 0.09\%$ after all fees. Approximately 20% of discovered paths are intra-exchange single trades exploiting minor stablecoin depegging, while the remaining 80% are genuine cross-exchange arbitrage routes. Opportunities persist across all 96 snapshots of the 8-hour overnight campaign. Sensitivity analysis confirms robustness to hyperparameter choices and linear profit scaling with order size (\$0.09 at \$100 to \$86 at \$100k). Graph scaling experiments (4–12 exchanges) show that A*-based methods scale monotonically while the 3-hop enumeration baseline exhibits high variance.

Quote staleness analysis demonstrates structural path persistence (99.6% survival at 300 s delay), though we note that CCXT-level ticker caching may mask true price-change

is this not h_3 ??

risk, and a historical tick-data replay would provide stronger evidence.

6.1. Future Work

Several directions emerge from our findings. First, *decoupling heuristic data acquisition from search*: h_2 's strong pruning but poor latency suggests an architecture where order-book data is pre-fetched asynchronously, enabling h_2 -quality guidance at h_4 -class latency. Second, extending the framework to *decentralized exchanges (DEXs)* and automated market makers (AMMs), where slippage is deterministic via bonding curves and can be precisely modelled within the heuristic. Third, *historical tick-data replay* to rigorously quantify price-change risk for the thin margins observed. Finally, exploring *adaptive heuristic selection*: since h_2 excels at search pruning and h_4 excels at latency, a system that dynamically selects the heuristic based on available data freshness and execution deadline could achieve the best of both.

6.2. Limitations

The current framework has not executed live trades on the blockchain for the presented data. Although we use real market pricing and fees, the profitability of a planned path may change between planning and execution. In practice, stablecoin markets rebalance under execution pressure, and order-book depth can be consumed by other market participants during the execution window.

Approximately 20% of discovered “arbitrage” paths are intra-exchange single trades that do not require cross-exchange transfers and are discoverable by trivial price comparisons. While these are real profit opportunities, they do not exercise the cross-exchange pathfinding that is the focus of this work.

Additionally, our exchange reliability scores are static priors based on historical uptime and community sentiment, which may not capture sudden exchange failures or regulatory actions. Future work should explore real-time reliability estimation from on-chain and off-chain signals.

References

- yaggy good job mr. smarky pants
- [1] R. Oantă and A. Coroiu. Crypto Advisor: A Web Application for Spotting Cross-Exchange Cryptocurrency Arbitrage Opportunities. In *Proceedings of the 15th International Conference on Computer Supported Education (CSEDU 2023)*, volume 1, pages 238–246, 2023.
 - [2] A. Shynkevich. Bitcoin arbitrage. *Finance Research Letters*, 40:101698, 2021.
 - [3] L. Zhou, K. Qin, C. F. Torres, D. V. Le, and A. Gervais. On the Just-In-Time Discovery of Profit-Generating Transactions in DeFi. In *IEEE Symposium on Security and Privacy (Oakland)*, 2021.
 - [4] Anonymous. An Improved Algorithm to Identify More Arbitrage Opportunities. *arXiv preprint arXiv:2406.16573*, 2024.
 - [5] Anonymous. Real-Time Identification of Negative Cycles for High-Efficiency Arbitrage. *VLDB*, 18:4081–4094, 2025.
 - [6] M. Kearns and Y. Nevmyvaka. Reinforcement Learning for Optimized Trade Execution. In *International Conference on Machine Learning (ICML)*, 2006.
 - [7] Anonymous. Optimal Execution with Reinforcement Learning. *arXiv preprint arXiv:2411.06389*, 2024.
 - [8] F. Fang, C. Ventre, M. Basios, L. Kanthan, D. Martinez-Rego, F. Wu, and L. Li. Cryptocurrency trading: a comprehensive survey. *Financial Innovation*, 8(1):1–59, 2022.
 - [9] K. Jothi and S. Oswalt Manoj. A comprehensive survey on blockchain and cryptocurrency technologies: Approaches, challenges, and opportunities. In *Blockchain, Artificial Intelligence, and the Internet of Things: Possibilities and Opportunities*, pages 1–22, 2022.
 - [10] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
 - [11] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.
 - [12] R. Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.
 - [13] L. R. Ford. Network flow theory. Technical report, RAND Corporation, 1956.
 - [14] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.
 - [15] CCXT. CryptoCurrency eXchange Trading Library. <https://github.com/ccxt/ccxt>, 2024. 9
 - [16] Python Software Foundation. Python Language Reference, version 3.12. <https://www.python.org/>, 2024.
 - [17] A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, 2008.
 - [18] Streamlit Inc. Streamlit: The fastest way to build and share data apps. <https://streamlit.io/>, 2024.
 - [19] J. D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
 - [20] W. McKinney. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference (SciPy 2010)*, pages 56–61, 2010.

- [21] E. Nikolova and D. R. Karger. Route Planning under Uncertainty: The Canadian Traveller Problem Revisited. In *Proceedings of the 23rd National Conference on Artificial Intelligence (AAAI)*, pages 969–974, 2008.
- [22] S. H. Lim, I. Rhee, and M. C. Fu. Risk-averse shortest path problems. *Mathematics of Operations Research*, 47(3):2001–2029, 2022.
- [23] A. Obadia, A. Salles, L. Sanchez, T. Sri, V. Buterin, and P. Daian. Unity is Strength: A Formalization of Cross-Domain Maximal Extractable Value. *arXiv preprint arXiv:2112.01472*, 2021.
- [24] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels. Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability. In *IEEE Symposium on Security and Privacy (Oakland)*, 2020.
- [25] I. Pohl. Heuristic search viewed as path finding in a graph. *Artificial Intelligence*, 1(3-4):193–204, 1970.

A. Appendix: Interactive Visualization System

A.1. Streamlit User Interface

We provide an interactive web-based interface built with Streamlit that enables real-time exploration of arbitrage opportunities and heuristic-guided pathfinding algorithms. The UI integrates live market data from multiple exchanges via the CCXT library, providing immediate visualization of the arbitrage graph structure and execution of search algorithms with real-time logging.

A.2. System Components

The interface consists of four primary tabs: (1) **Arbitrage Graph** displays an interactive network visualization where nodes represent (exchange, stablecoin) pairs and edges encode trades (blue) and transfers (green), with controls for heuristic selection, starting wallet, and capital allocation; (2) **Live Prices** shows real-time stablecoin prices across exchanges fetched via CCXT APIs; (3) **Fees** visualizes all fee structures used in the model; (4) **How to Use** provides documentation.

A.3. Real-Time Data Integration

The system uses CCXT to fetch live market data including ticker prices (bid/ask), 24-hour trading volume, and order book depth. Upon initialization or manual refresh, prices are normalized to USD, filtered for stablecoin peg compliance (5% tolerance), and used to build the weighted graph structure. All data fetching occurs synchronously to ensure search algorithms operate on consistent market snapshots.

A.4. Interactive Algorithm Execution

Users can execute all four heuristic-guided search algorithms through a unified interface. During execution, a status container displays streaming log messages showing heuristic initialization, node expansion order, intermediate profitable paths, and final optimal path selection. Upon completion, results include start/final cash amounts, complete route paths, per-node heuristic values with risk assessments, and step-by-step execution details including fees and transfer times.

A.5. Technical Implementation

The UI uses Streamlit for web rendering, NetworkX for graph structures, Matplotlib for visualization, and Pandas for tabular display. Real-time logging is achieved through custom `logging.Handler` subclasses that update Streamlit containers asynchronously. CCXT provides a unified interface to multiple exchange APIs with graceful error handling for API failures and network timeouts.